

SOFTWARE ENGINEERING COURSE

LECTURE 6

Software Architectural Design

Micheal Tuape, D.Sc



MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Development Life Cycle & Methodologies	Ch. 2
Week 4	September 23 — September 28, 2024	Software Development Processes	Ch. 3
Week 5	September 30 — October 4, 2024	Requirements Engineering	Ch. 4
Week 6	October 7 — October 11, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 14 — October 18, 2024	Software Architectural Design	Ch. 6





LEARNING OBJECTIVES

- »» Architectural design decisions
- »» Architectural views
- »» Architectural patterns
- »» Application architectures





ARCHITECTURAL DESIGN

- »» Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- »» Architectural design is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them.
- »» The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.





AGILITY AND ARCHITECTURE

- »» It is generally accepted that an early stage of agile processes is to design an overall systems architecture.
- »» Refactoring the system architecture is usually expensive because it affects so many components in the system





ARCHITECTURAL ABSTRACTION

- »» Architecture in the small is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components.
- »» Architecture in the large is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.





ADVANTAGES OF EXPLICIT ARCHITECTURE

- »» Stakeholder communication
 - Architecture may be used as a focus of discussion by system stakeholders.
- »» System analysis
 - Means that analysis of whether the system can meet its non-functional requirements is possible.
- »» Large-scale reuse
 - The architecture may be reusable across a range of systems
 - Product-line architectures may be developed.





BOX AND LINE DIAGRAMS

- » Simple, informal block diagrams showing entities and relationships are the most frequently used method for documenting software architectures.
- » But these have been criticised because they lack semantics, do not show the types of relationships between entities nor the visible properties of entities in the architecture.
- » Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- » However, useful for communication with stakeholders and for project planning.





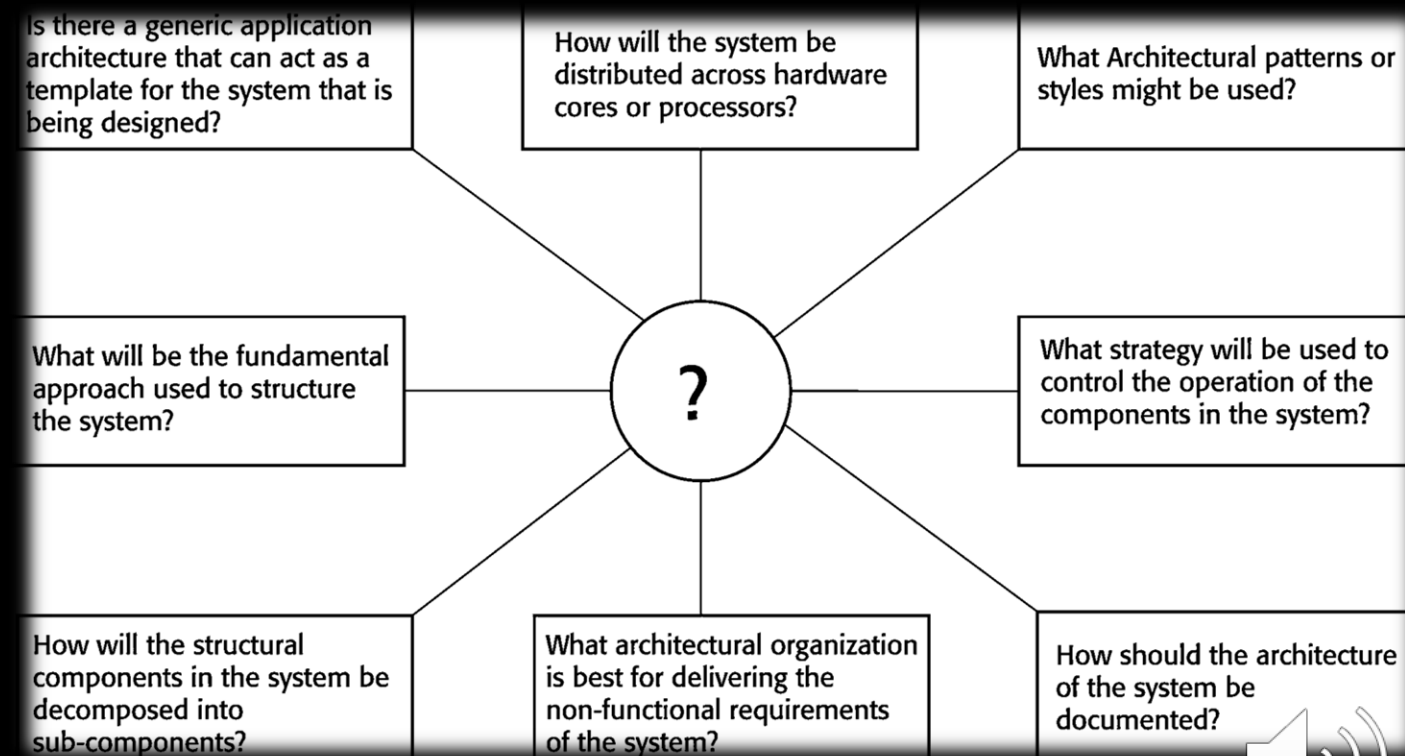
USE OF ARCHITECTURAL MODELS

- »» As a way of facilitating discussion about the system design
 - A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system without being confused by detail.
- »» As a way of documenting an architecture that has been designed
 - The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.



ARCHITECTURAL DESIGN DECISIONS

- Architectural design is a creative process so the process differs depending on the type of system being developed.
- However, a number of common decisions span all design processes and these decisions affect the non-functional characteristics of the system.





ARCHITECTURE REUSE

- »» Systems in the same domain often have similar architectures that reflect domain concepts.
- »» Application product lines are built around a core architecture with variants that satisfy particular customer requirements.
- »» The architecture of a system may be designed around one of more architectural patterns or 'styles'.
 - These capture the essence of an architecture and can be instantiated in different ways.





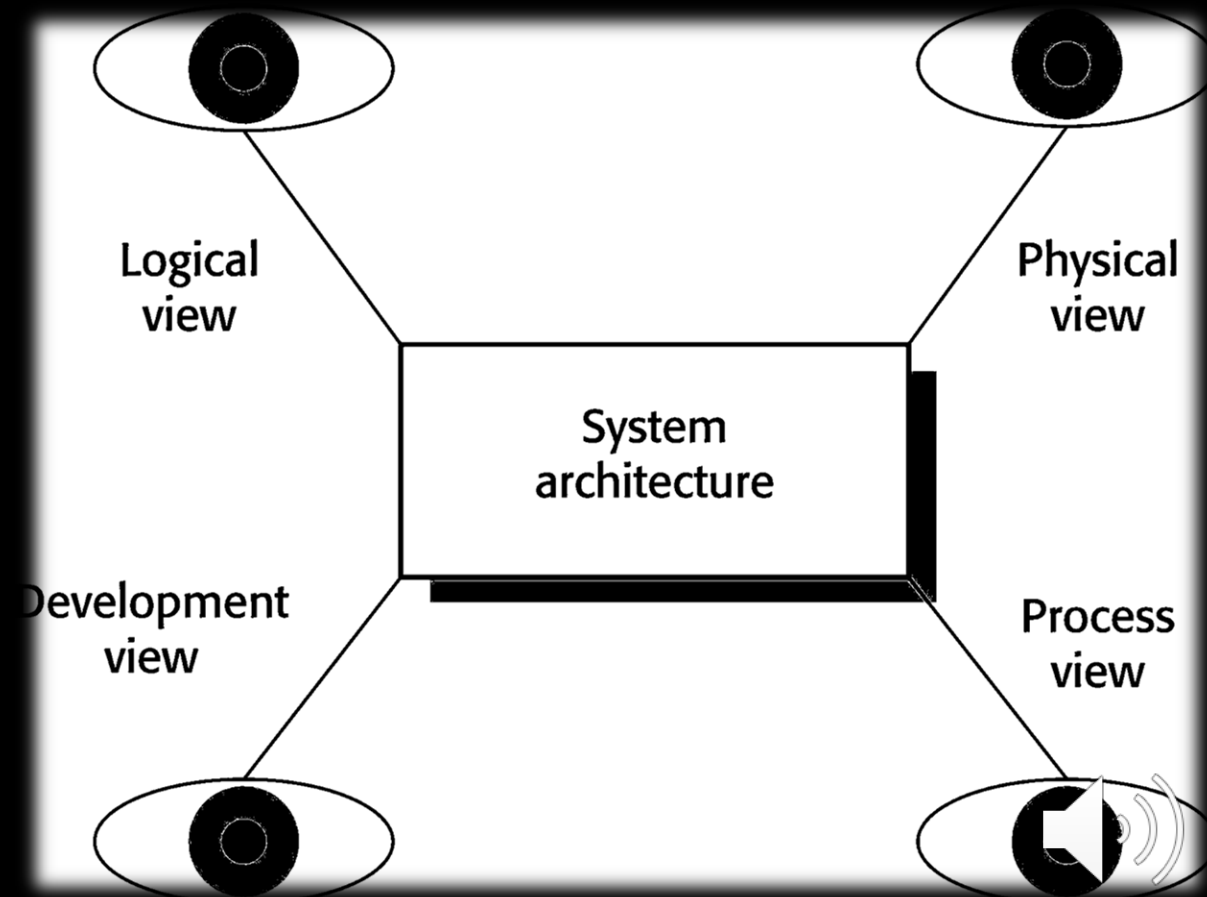
ARCHITECTURE AND SYSTEM CHARACTERISTICS

- »» **Performance:** Localise critical operations and minimise communications. Use large rather than fine-grain components.
- »» **Security:** Use a layered architecture with critical assets in the inner layers.
- »» **Safety:** Localise safety-critical features in a small number of sub-systems.
- »» **Availability:** Include redundant components and mechanisms for fault tolerance.
- »» **Maintainability:** Use fine-grain, replaceable components.



ARCHITECTURAL VIEWS

- » What views or perspectives are useful when designing and documenting a system's architecture?
- » What notations should be used for describing architectural models?
- » Each architectural model only shows one view or perspective of the system.



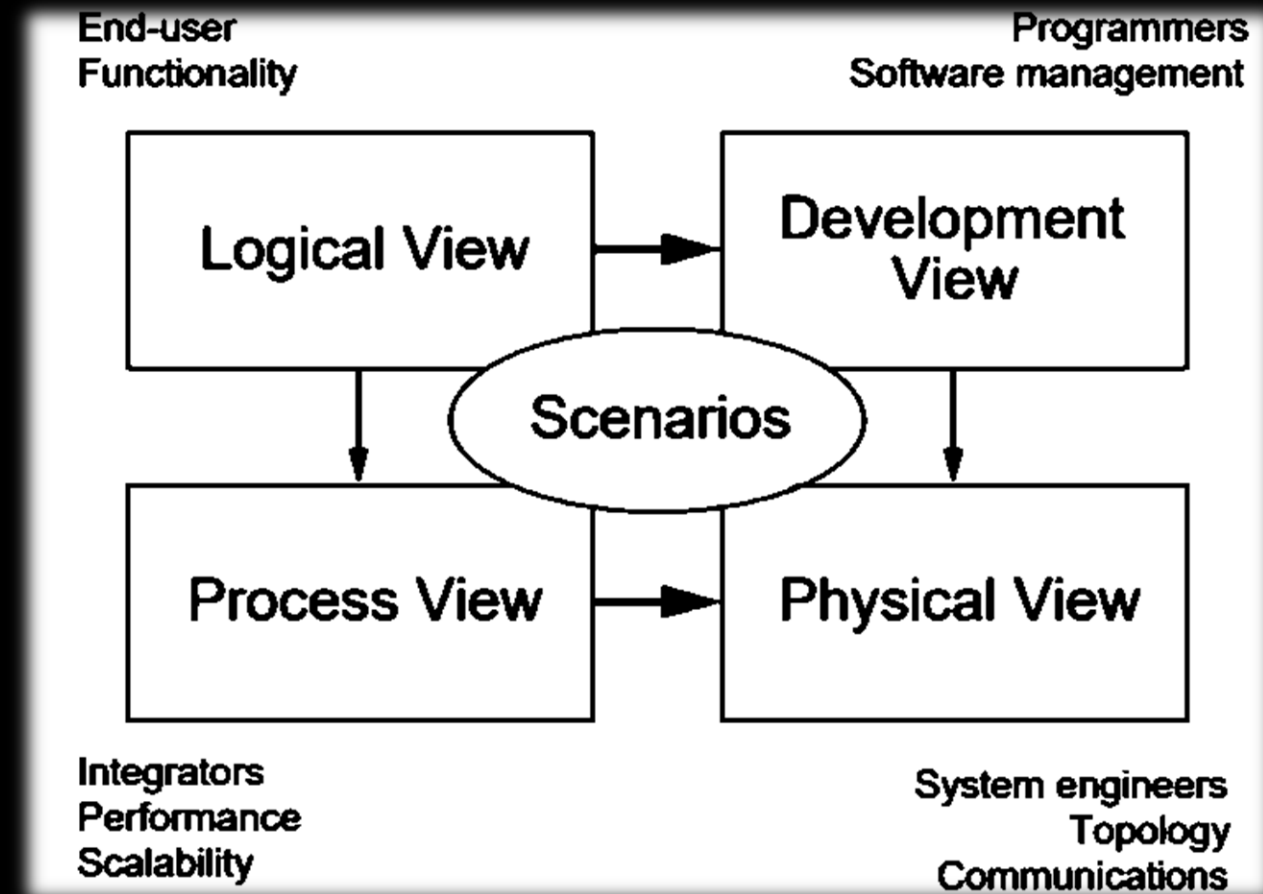


REPRESENTING ARCHITECTURAL VIEWS

- »» Some people argue that the Unified Modeling Language (UML) is an appropriate notation for describing and documenting system architectures
- »» Architectural description languages (ADLs) have been developed but are not widely used



MORE ON ARCHITECTURAL VIEWS



MORE ON ARCHITECTURAL VIEWS-LOGICAL VIEW NOTATIONS

The logical architecture primarily supports the functional requirements—what the system should provide in terms of services to its users. The system is decomposed into a set of key abstractions, taken (mostly) from the problem domain, in the form of objects or object classes

Components



Class



Class Utility



Parameterized Class



Class category

Connectors



Association



Containment, Aggregation



Usage



Inheritance

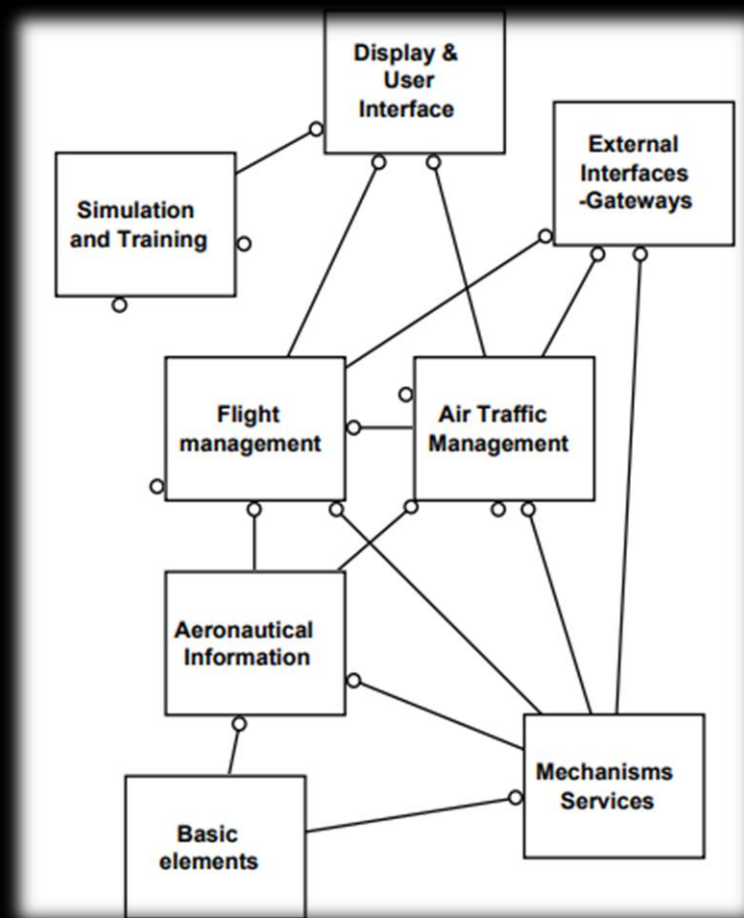


Instanciation



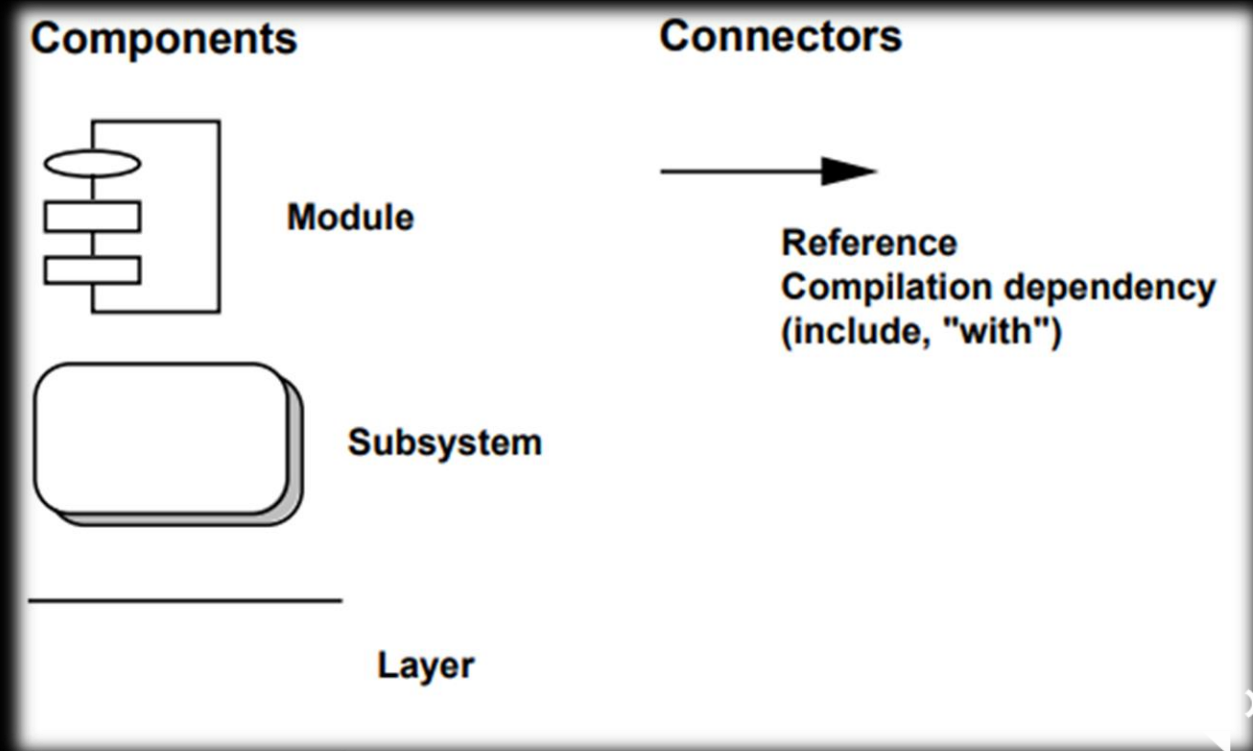
MORE ON ARCHITECTURAL VIEWS-LOGICAL VIEW

Blueprint for an Air Traffic Control System



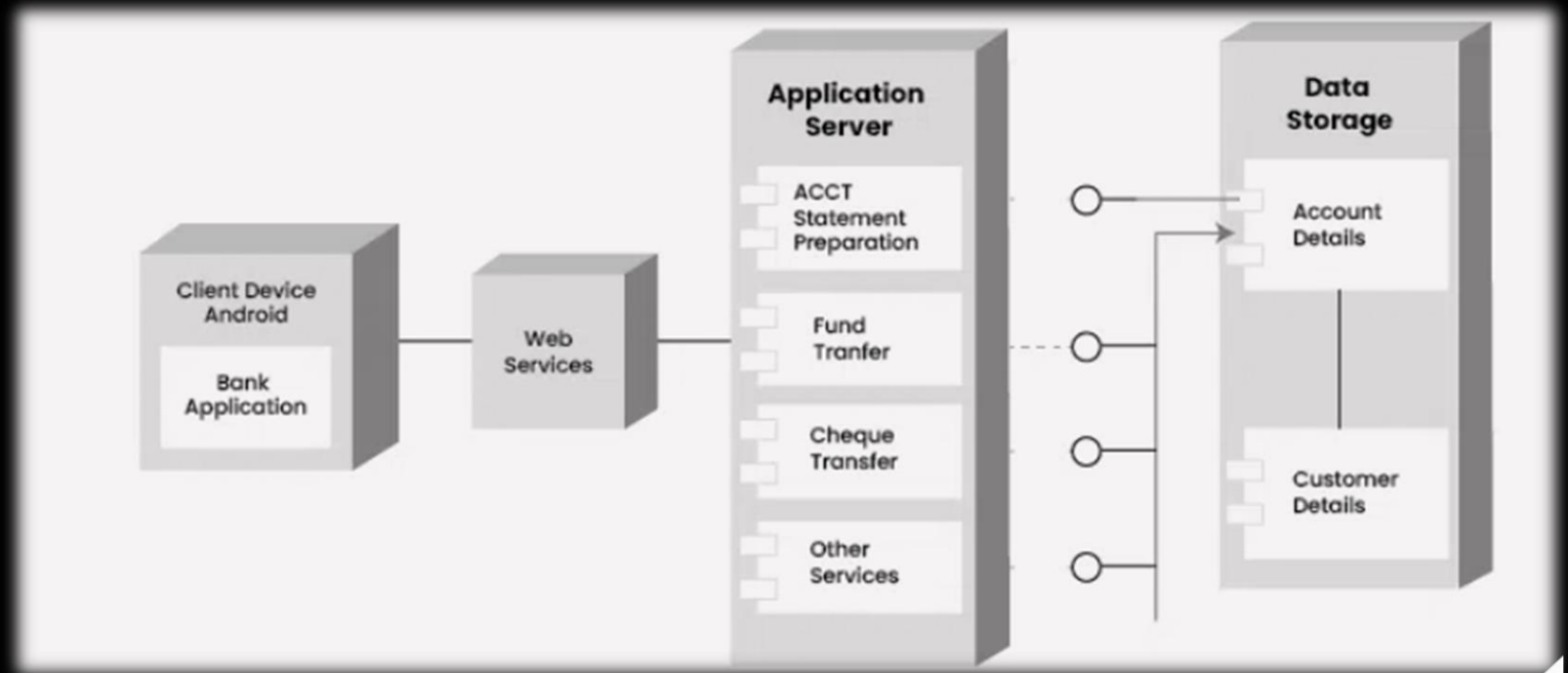
ARCHITECTURAL VIEWS DEVELOPMENT VIEW NOTATION

The development architecture focuses on the actual software module organization on the software development environment. The software is packaged in small chunks—program libraries, or subsystems—that can be developed by one or a small number of developers. T



ARCHITECTURAL VIEWS DEVELOPMENT VIEW

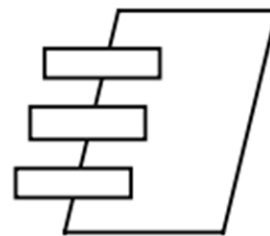
Subsystem decomposition



ARCHITECTURAL VIEWS-PROCESS VIEWS NOTATIONS

The process architecture takes into account some non-functional requirements, such as performance and availability. It addresses issues of concurrency and distribution, of system's integrity, of fault-tolerance, and how the main abstractions from the logical view fit within the process architecture—on which thread of control is an operation for an object actually executed.

Components



Process



Simplified Process



Periodic process adornment

Connectors



Unspecified



Message



Remote Procedure Call



Message, bidirectional

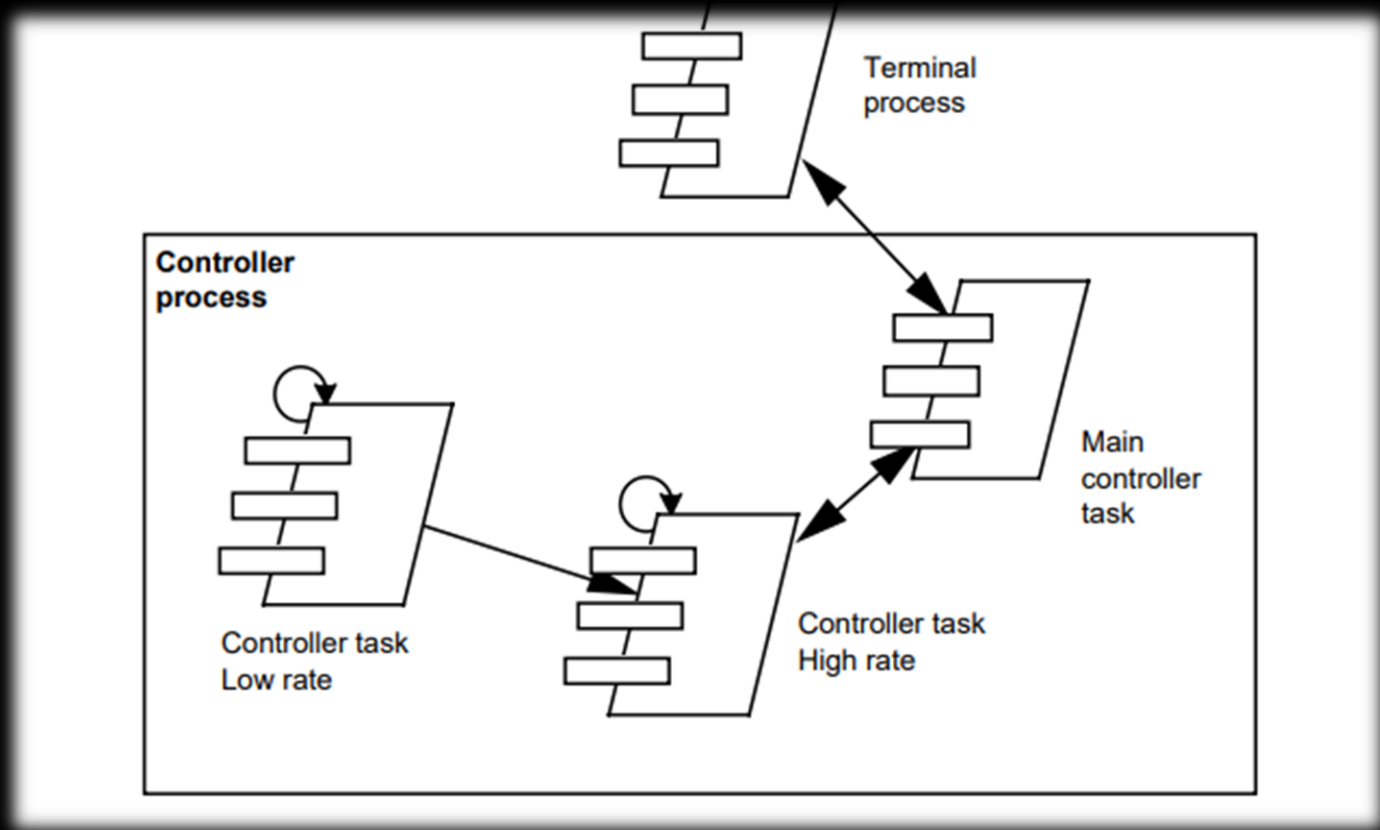


Event broadcast

(Indicates a cyclical process)

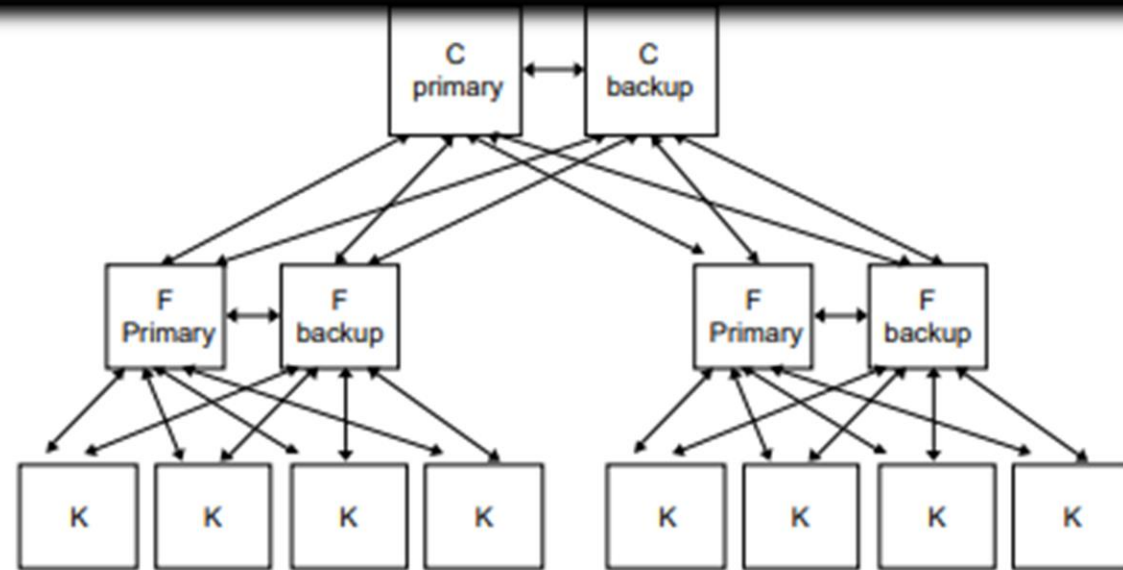


ARCHITECTURAL VIEWS-PROCESS VIEWS



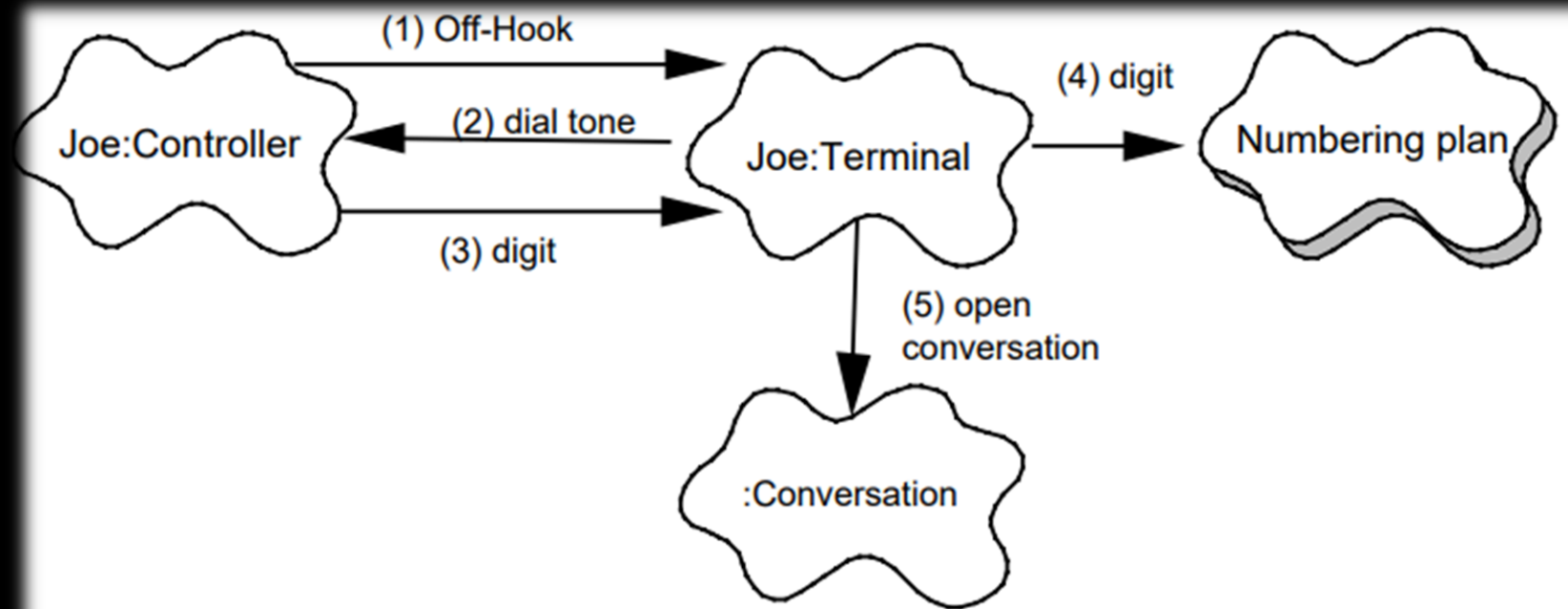
ARCHITECTURAL VIEWS- PHYSICAL VIEWS

The physical architecture takes into account primarily the non-functional requirements of the system such as availability, reliability (fault-tolerance), performance (throughput), and scalability. The software executes on a network of computers, or processing nodes (or just nodes for short).



MORE ON ARCHITECTURAL VIEWS SCENARIOS

A scenario for a
local call—selection
phase





ARCHITECTURAL PATTERNS

- »» Patterns are a means of representing, sharing and reusing knowledge.
- »» An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.
- »» Patterns should include information about when they are and when they are not useful.
- »» Patterns may be represented using tabular and graphical descriptions.



THE MODEL-VIEW-CONTROLLER (MVC) PATTERN

NAME	MVC (MODEL-VIEW-CONTROLLER)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction and passes these interactions to the View and the Model.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.





LAYERED ARCHITECTURE

- »» Used to model the interfacing of sub-systems.
- »» Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- »» Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- »» However, often artificial to structure systems in this way.



THE LAYERED ARCHITECTURE PATTERN

NAME	LAYERED ARCHITECTURE
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest-level layers represent core services that are likely to be used throughout the system.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.





A GENERIC LAYERED ARCHITECTURE

User interface

User interface management
Authentication and authorization

Core business logic/application functionality
System utilities

System support (OS, database, etc.)





REPOSITORY ARCHITECTURE

- »» Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems;
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- »» When large amounts of data are to be shared, the repository model of sharing is most commonly used as this is an efficient data sharing mechanism.



THE REPOSITORY PATTERN

NAME	REPOSITORY
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent-they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.



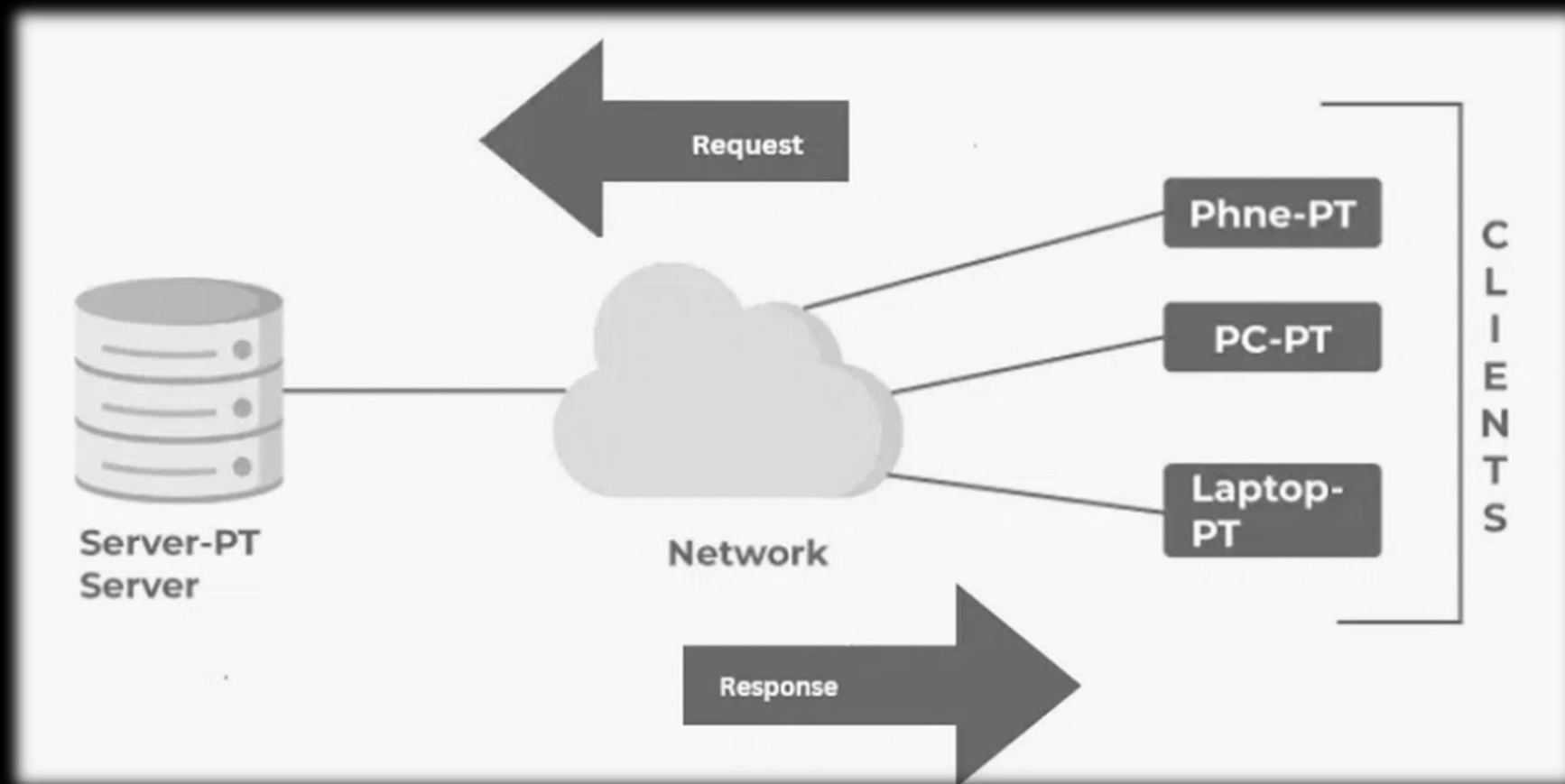


CLIENT-SERVER ARCHITECTURE

- »» Distributed system model which shows how data and processing is distributed across a range of components.
 - Can be implemented on a single computer.
- »» Set of stand-alone servers which provide specific services such as printing, data management, etc.
- »» Set of clients which call on these services.
- »» Network which allows clients to access servers.



CLIENT-SERVER ARCHITECTURE



THE CLIENT–SERVER PATTERN

NAME	CLIENT-SERVER
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.



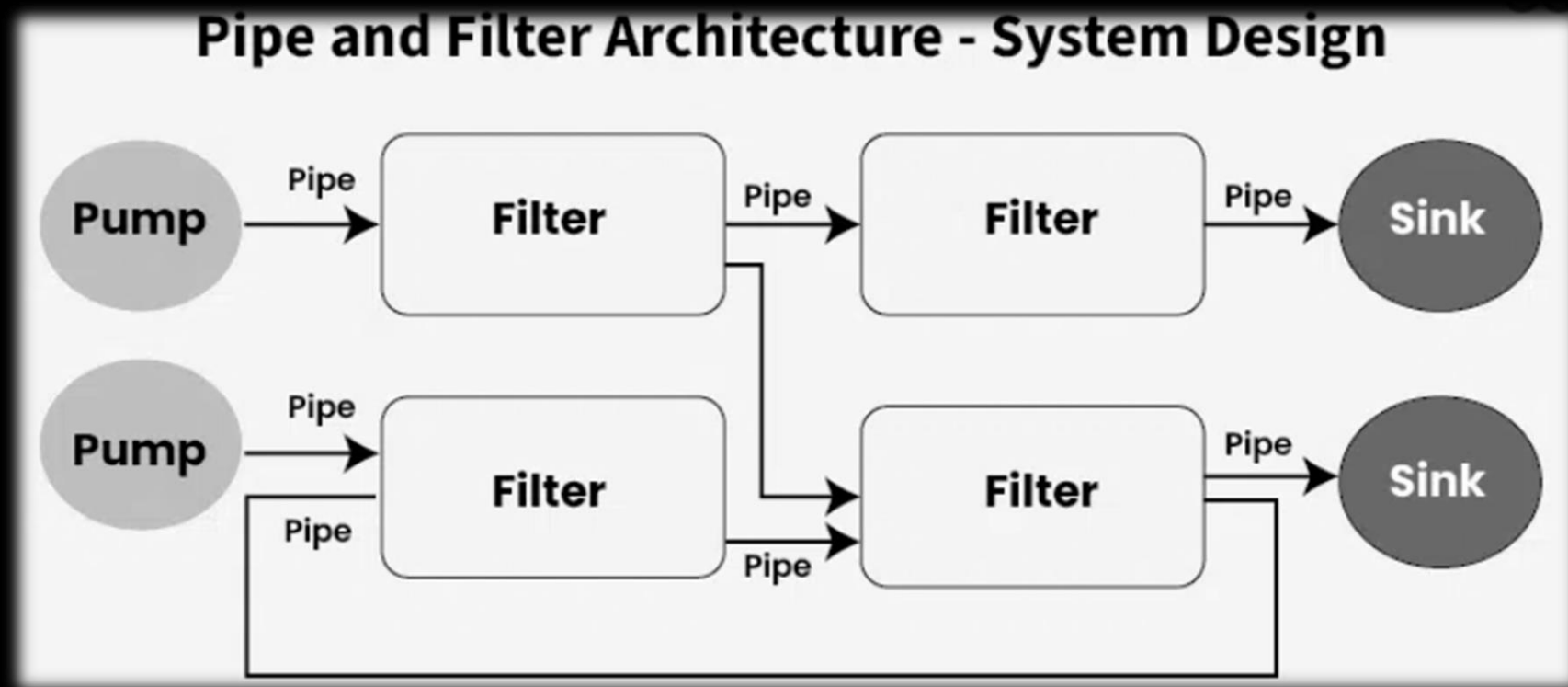


PIPE AND FILTER ARCHITECTURE

- »» Functional transformations process their inputs to produce outputs.
- »» May be referred to as a pipe and filter model (as in UNIX shell).
- »» Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- »» Not really suitable for interactive systems.



PIPE AND FILTER ARCHITECTURE



THE PIPE AND FILTER PATTERN

NAME	PIPE AND FILTER
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
When used	Commonly used in data processing applications where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.





APPLICATION ARCHITECTURES

- »» Application systems are designed to meet an organisational need.
- »» As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements.
- »» A generic application architecture is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements.





USE OF APPLICATION ARCHITECTURES

- »» As a starting point for architectural design.
- »» As a design checklist.
- »» As a way of organising the work of the development team.
- »» As a means of assessing components for reuse.
- »» As a vocabulary for talking about application types.





INFORMATION SYSTEMS ARCHITECTURE

- »» Information systems have a generic architecture that can be organised as a layered architecture.
- »» These are transaction-based systems as interaction with these systems generally involves database transactions.
- »» Layers include:
 - The user interface
 - User communications
 - Information retrieval
 - System database





LAYERED INFORMATION SYSTEM ARCHITECTURE

User interface

User communications

Authentication and
authorization

Information retrieval and modification

Transaction management

Database



LAYERED ARCHITECTURE

Presentation Layer – responsible for user interactions with the software system

Application/Business Layer – handles aspects related to accomplishing functional requirements

Domain Layer – responsible for algorithms, and programming components

Infrastructure/Persistence/Database Layer – responsible for handling data, databases

